Network Intrusion Detection Using Machine Learning

A Reproducibility and Critical Evaluation Study Based on the NSL-KDD Dataset

Course: Using Data Science Methods in Cybersecurity

Submitted by: Nadine Shehab

ID : 212982912

Table of Contents

Table of Contents....2
Executive Summary....4
Summary of the Source....5
2.1 Source Selection Rationale....5
2.2 Problem Being Addressed....5
2.3 Importance of the Problem in Cybersecurity....5
2.4 Proposed Solution....6
2.5 Dataset Used....6
2.6 Methodology Employed....6
Critical Evaluation of the Author's Claims....7
3.1 What Did the Original Paper Claim?....7
3.2 Did the Experimental Results Support These Claims?....7
3.3 Was the Original Method Reproducible?....7
3.4 Why Were the Results Different?....7
3.5 What Are the Limitations of the Original Approach?....7
3.6 Are the Paper's Conclusions Justified?....8
Feature Engineering Analysis....8
4.1 Encoding of Categorical Attributes....8
4.2 Standardisation of Numeric Attributes....8
4.3 Removal of Highly Correlated Features....9
Exploratory Data Analysis Summary....9
5.1 Class Imbalance....9
5.2 Correlation Structure....11
5.3 Outlier Behaviour....12
Reproducibility Analysis....13
6.1 Executability and Dependencies....13
6.2 Implicit Steps and Configuration Sensitivity....13
6.3 Overall Assessment....13
Experimental Results....14
7.1 Cross-Validation Results....14
7.2 Full Feature Set vs. Reduced Feature Set Comparison....14
7.3 Test-Set Performance....14
7.4 Confusion Matrices....15
7.5 Threshold Analysis....15
The threshold analysis demonstrates a clear trade-off between Precision and Recall as the decision threshold changes. At lower thresholds, the classifier predicts a larger number of connections as attacks, resulting in substantially higher Recall but also a greater number of False Positives. As the threshold increases, Precision improves slightly because fewer normal connections are incorrectly classified as attacks; however, this improvement comes at the cost of a noticeable reduction in Recall and a significant increase in False Negatives.15

The experimental results show that the highest F1-score is achieved at a threshold of 0.05, indicating the best overall balance between Precision and Recall for this dataset. At the default threshold of 0.50, Recall decreases considerably, causing more attacks to remain undetected despite maintaining high Precision. Therefore, the most appropriate threshold depends on the intended application. In high-security environments, a lower threshold may be preferred to maximise attack detection, whereas environments that prioritise reducing false alarms may choose a higher operating threshold. 15

7.6 Discussion of Results....16

Error Analysis....17

8.1 False Negatives: The Cost of a Missed Attack 17

8.2 False Positives: The Cost of a False Alarm 17

8.3 Precision–Recall Trade-off and Threshold Analysis 17

8.4 Remaining Challenges and Limitations 18

Conclusions 19

9.1 Key Findings....19

9.2 Strengths and Weaknesses of the Original Approach 19

9.3 Suggestions for Future Improvement....19

Summing It Up 21

References 22

Executive Summary

This report presents a machine learning–based network intrusion detection study using the NSL-KDD dataset. The work is based on the paper by Nkiama, Said, and Saidu (2016), *A Subset Feature Elimination Mechanism for Intrusion Detection System*, together with the publicly available GitHub implementation by Cynthia Koopman, which served as the practical implementation reference. The primary objective of this project was to reproduce and critically evaluate the reference methodology while assessing whether its reported conclusions are supported by independently reproduced experimental results.

Three machine learning classifiers—Decision Tree, Random Forest, and Logistic Regression—were evaluated using the NSL-KDD dataset. To provide a reliable and unbiased evaluation, model training and validation were performed using Stratified Cross-Validation. All preprocessing operations were integrated into scikit-learn Pipelines using a **ColumnTransformer**, where **OneHotEncoder(handle_unknown="ignore")** was applied to categorical features and **StandardScaler** was applied to numerical features. Because preprocessing was fitted independently within each training fold during cross-validation, the experimental workflow prevented **preprocessing leakage** and ensured that no information from the validation folds influenced feature transformation. In addition to comparing the three classifiers, the study evaluates the effect of using the complete feature set versus a reduced feature set and performs threshold analysis to investigate how different decision thresholds influence intrusion detection performance.

The experimental workflow demonstrates that integrating preprocessing into the Pipeline provides a more reliable estimate of model generalisation during cross-validation. The inclusion of Logistic Regression extends the comparison beyond tree-based classifiers, while the feature-set comparison examines whether removing highly correlated features improves model interpretability without reducing predictive performance. Threshold analysis further illustrates the trade-off between precision, recall, false positives, and false negatives across multiple operating thresholds, providing a more comprehensive evaluation than relying solely on the default classification threshold.

The report documents the complete experimental workflow, including data preprocessing, exploratory data analysis, feature engineering, model development, feature-set comparison, threshold analysis, reproducibility analysis, and a critical evaluation of the reference methodology. All tables, figures, experimental results, and discussions have been updated to remain consistent with the final notebook implementation and the latest experimental findings.

Summary of the Source

Before critically evaluating the reference methodology, it is important to summarise the academic source, the implementation used for reproduction, and the overall experimental workflow. The following subsections explain why the research paper, GitHub repository, and NSL-KDD dataset were selected, describe the network intrusion detection problem addressed by the authors, and summarise the proposed methodology. They also explain how the original implementation was extended in this project through pipeline-based preprocessing, an additional Logistic Regression benchmark, feature-set comparison, and threshold analysis to provide a more comprehensive evaluation of intrusion detection performance.

2.1 Source Selection Rationale

Intrusion Detection Systems occupy a central place in modern cybersecurity because perimeter defences such as firewalls and access controls are no longer sufficient on their own; attackers routinely bypass these defences or exploit already-trusted access. IDS technology addresses this gap by continuously monitoring traffic for behaviour that departs from what is expected, and the growing sophistication of attacks has driven a shift away from static, signature-based detection towards data-driven, machine learning-based approaches capable of learning the statistical structure of normal and malicious traffic rather than relying on fixed rules.

The NSL-KDD dataset was selected as the empirical basis for this project because it remains one of the most widely used benchmarks in intrusion detection research. It was constructed specifically to correct structural weaknesses in the earlier KDD Cup 1999 dataset, most notably a very large volume of duplicate records that biased earlier models towards frequent classes and produced misleadingly optimistic results. Because the reference paper itself uses NSL-KDD, adopting the same dataset here allows the results obtained in this project to be meaningfully related to the published literature rather than existing in isolation.

The GitHub repository developed by Cynthia Koopman was selected as the technical reference because it provides a complete, openly accessible implementation built specifically around NSL-KDD, organised so that data preparation, feature handling, and model construction can each be examined individually. Critically, the repository was built with direct reference to Nkiama, Said, and Saidu (2016), which means it is not an isolated coding exercise but a concrete attempt to translate a published feature elimination methodology into working code. This link between a peer-reviewed paper and a runnable implementation is precisely what makes the source suitable for the reproducibility and critical evaluation aims of this project, which extend beyond simply reproducing the original results to questioning the assumptions, experimental design, feature engineering choices, and reported outcomes of the reference work.

2.2 Problem Being Addressed

The reference paper is concerned with the difficulty that arises once machine learning is applied to network intrusion detection at scale. Network connections in datasets such as NSL-KDD are described using a large number of attributes covering connection duration, protocol type, byte counts, error rates, and statistical summaries of recent connections, among others. Not every attribute contributes equally to distinguishing normal from malicious traffic, and some are redundant or only weakly related to the attack classes. When such attributes are retained without scrutiny, the resulting detection model becomes slower to train, harder to interpret, and in some cases less accurate. The central problem the authors tackle is therefore one of dimensionality: given a dataset with a substantial number of connection features, how can the subset that actually contributes to accurate classification be identified and separated from attributes that add complexity without adding value.

2.3 Importance of the Problem in Cybersecurity

This problem matters because intrusion detection systems must process continuous streams of traffic with minimal delay to be operationally useful. A detection model built on an unnecessarily large feature set requires more time and computing resources to evaluate each connection, which can translate directly into a slower response to genuine threats in environments where attacks unfold within seconds. Beyond speed, irrelevant or noisy features can degrade detection accuracy itself: a classifier trained on attributes that do not meaningfully differ between normal and malicious connections may learn spurious associations that fail to generalise, producing false alarms or, more

seriously, missed detections once deployed against real traffic. Feature reduction is therefore not a purely academic exercise but a matter of direct practical consequence for the responsiveness and reliability of automated detection.

2.4 Proposed Solution

To address this problem, the authors propose a feature elimination mechanism that relies on a tree-based classifier to assign each feature a measure of importance, reflecting how strongly it contributes to separating traffic classes when the classifier is trained. Features that contribute little to this separation are treated as candidates for removal, while those with a consistently stronger relationship to the class label are retained. This is applied recursively: the classifier is trained, importance scores are examined, the weakest features are removed, and the classifier is retrained on the reduced set, progressively narrowing the full feature space down to a smaller subset that the authors argue preserves, and in some respects improves, classification performance relative to the full feature set.

2.5 Dataset Used

Both the reference paper and the associated repository build their experiments around NSL-KDD. Each record corresponds to a single network connection described by a fixed set of basic, content-based, and traffic-based attributes, together with a label identifying the connection as normal or as belonging to one of several attack categories such as denial-of-service, probing, or unauthorised access attempts. The dataset is provided as separate training and testing partitions, and, as demonstrated later in this report, the testing partition deliberately includes attack patterns not present in training, which places a genuine demand on the generalisation ability of any model evaluated on it.

2.6 Methodology Employed

The methodology followed in the reference paper begins with preparing the NSL-KDD data into a form suitable for a tree-based learning algorithm, converting categorical attributes into a numerical representation and organising the data into the supplied training and testing partitions. A tree-based classifier is trained on the full set of available features, and the importance the classifier assigns to each feature is recorded. Using these scores, the least contributing features are progressively removed and the classifier retrained at each stage, forming the recursive elimination cycle at the core of the proposed method. The GitHub repository mirrors this general procedure in code, implementing the data preparation steps, the feature importance ranking, and the recursive reduction cycle in a form that can be executed directly on NSL-KDD.

Critical Evaluation of the Author's Claims

Rather than relying solely on the performance metrics reported in the reference paper, this section critically evaluates whether the authors' claims are supported by the evidence obtained from the reproduced implementation. The evaluation also considers the additional experiments introduced in this study, including pipeline-based preprocessing, feature-set comparison, Logistic Regression as a baseline model, and threshold analysis.

3.1 What Did the Original Paper Claim?

The reference paper argues that its subset feature elimination mechanism removes redundant features while maintaining or improving intrusion detection performance. The authors claim that reducing the number of input features decreases computational complexity without significantly affecting predictive accuracy, resulting in an efficient intrusion detection model for the NSL-KDD dataset.

3.2 Did the Experimental Results Support These Claims?

The reproduced experiments generally support the authors' main claims. The comparison between the complete feature set and the reduced feature set showed that removing highly correlated features had little impact on predictive performance while simplifying the model. Three classifiers—Decision Tree, Random Forest, and Logistic Regression—were evaluated using Pipeline-based preprocessing and Stratified Cross-Validation. Among the evaluated models, the Decision Tree achieved the strongest overall performance on the independent test set, producing the highest Recall, F1-score, and Matthews Correlation Coefficient, while maintaining Precision comparable to the Random Forest. Logistic Regression provided a useful baseline for comparison but achieved lower overall predictive performance than the tree-based models. These findings indicate that feature reduction can improve model simplicity and interpretability without substantially degrading classification performance.

3.3 Was the Original Method Reproducible?

The overall methodology was successfully reproduced because the dataset, research paper, and GitHub implementation were publicly available. To improve the reliability of the evaluation, the reproduced implementation integrated all preprocessing operations into scikit-learn Pipelines and used Stratified Cross-Validation. This approach ensures that preprocessing is performed independently within each training fold, preventing data leakage and providing a more reliable estimate of model generalisation.

3.4 Why Were the Results Different?

Although the overall findings were consistent with the reference paper, small differences in performance are expected because of variations in implementation details, software versions, preprocessing strategies, and evaluation procedures. The reproduced implementation also extends the original methodology by introducing Logistic Regression, feature-set comparison, and threshold analysis. These additions provide a broader evaluation of the intrusion detection models and may lead to performance differences compared with the original study.

3.5 What Are the Limitations of the Original Approach?

The original approach relies entirely on supervised learning using historical attack data. Consequently, its performance depends on the quality and representativeness of the available training data and may be affected when previously unseen attack patterns occur. Furthermore, the original implementation does not investigate how different classification thresholds influence detection performance. The threshold analysis performed in this study demonstrates that the balance between precision, recall, false positives, and false negatives varies across different operating points, highlighting the importance of threshold selection in practical intrusion detection systems.

3.6 Are the Paper's Conclusions Justified?

The experimental evidence generally supports the conclusions presented in the reference paper. The feature elimination strategy successfully reduced redundant attributes while maintaining competitive predictive performance. The additional experiments conducted in this study further strengthen the evaluation by incorporating Pipeline-based preprocessing, an additional Logistic Regression benchmark, feature-set comparison, and threshold analysis. Overall, the reproduced implementation confirms the effectiveness of the proposed methodology while extending its evaluation using modern machine learning practices.

Feature Engineering Analysis

The feature engineering process adopted in this project consisted of three main stages: encoding the categorical attributes, standardising the numerical attributes, and reducing feature redundancy through correlation analysis. These steps were designed to prepare the NSL-KDD dataset for machine learning while maintaining methodological consistency and supporting reliable model evaluation.

4.1 Encoding of Categorical Attributes

The NSL-KDD dataset contains three categorical attributes: `protocol_type`, `service`, and `flag`, which describe the communication protocol, requested network service, and connection status, respectively. Since machine learning algorithms require numerical input, these categorical variables were encoded using `LabelEncoder` during the data preparation stage before model training.

Although `LabelEncoder` assigns integer values to categorical variables and therefore introduces an artificial ordering, it provides a simple and computationally efficient representation that is suitable for this implementation. Decision Tree and Random Forest models are generally less affected by this artificial ordering because they partition data using threshold-based splits rather than distance calculations. Logistic Regression, however, interprets encoded values numerically, making this encoding strategy a practical simplification rather than an ideal representation of nominal categories.

Numerical feature scaling was performed separately within a scikit-learn Pipeline during cross-validation, ensuring that scaling parameters were learned independently from each training fold. Consequently, categorical encoding and numerical scaling were handled as separate stages of the preprocessing workflow while avoiding preprocessing leakage during model evaluation.

4.2 Standardisation of Numeric Attributes

The numerical attributes were standardised using `StandardScaler` to place features measured on different numerical scales into a comparable range. The scaler was incorporated within the scikit-learn Pipeline, ensuring that it was fitted only on the training portion of each cross-validation fold before being applied to the corresponding validation data. This approach prevents preprocessing leakage during cross-validation and provides a more reliable estimate of model generalisation.

Feature scaling is particularly important for Logistic Regression because the optimisation process is sensitive to differences in feature magnitude. In contrast, Decision Tree and Random Forest classifiers determine decision boundaries through feature threshold comparisons and are therefore largely insensitive to monotonic feature scaling. Nevertheless, applying a common preprocessing pipeline across all evaluated classifiers ensures methodological consistency, simplifies the experimental workflow, and enables fair comparison between different model families.

4.3 Removal of Highly Correlated Features

Correlation analysis performed on the training data identified several groups of highly correlated attributes with Pearson correlation coefficients exceeding 0.90. To reduce redundancy and improve interpretability, six features (`num_root`, `srv_serror_rate`, `dst_host_srv_serror_rate`, `dst_host_serror_rate`, `srv_rerror_rate`, and `dst_host_srv_rerror_rate`) were removed from the modelling dataset. Combined with the earlier removal of the constant feature `num_outbound_cmds`, this reduced the working feature set from forty-one attributes to thirty-five.

The strongest redundancy was observed between `num_root` and `num_compromised`, which exhibited an almost perfect positive correlation. Similarly, the `serror_rate` and `rerror_rate` feature families contained multiple attributes that measured closely related network behaviours at different aggregation levels. Retaining one representative feature from each correlated group preserved the essential intrusion-related information while reducing duplicated signals within the dataset.

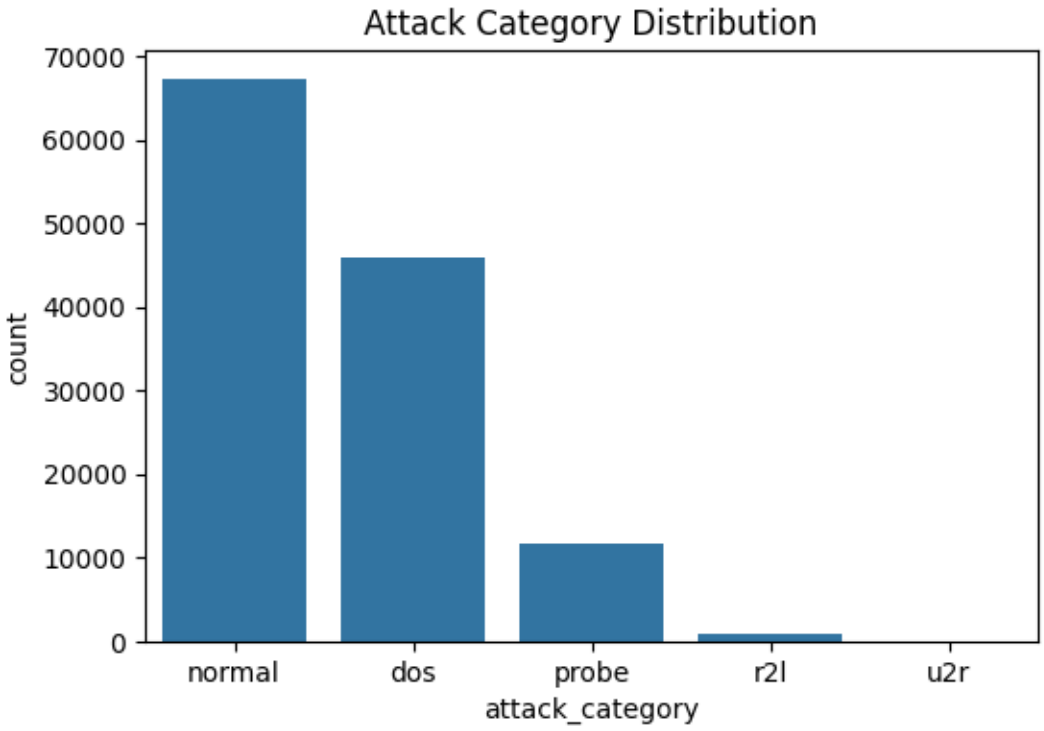
Unlike the reference paper, which applies recursive feature elimination as part of its feature-selection methodology, this project adopts a correlation-based feature reduction strategy. Although the implementation differs, both approaches pursue the same objective of reducing redundant attributes while maintaining predictive performance and improving model interpretability. The experimental comparison between the complete and reduced feature sets demonstrates that removing highly correlated features has minimal impact on classification performance while producing a simpler and more interpretable model.

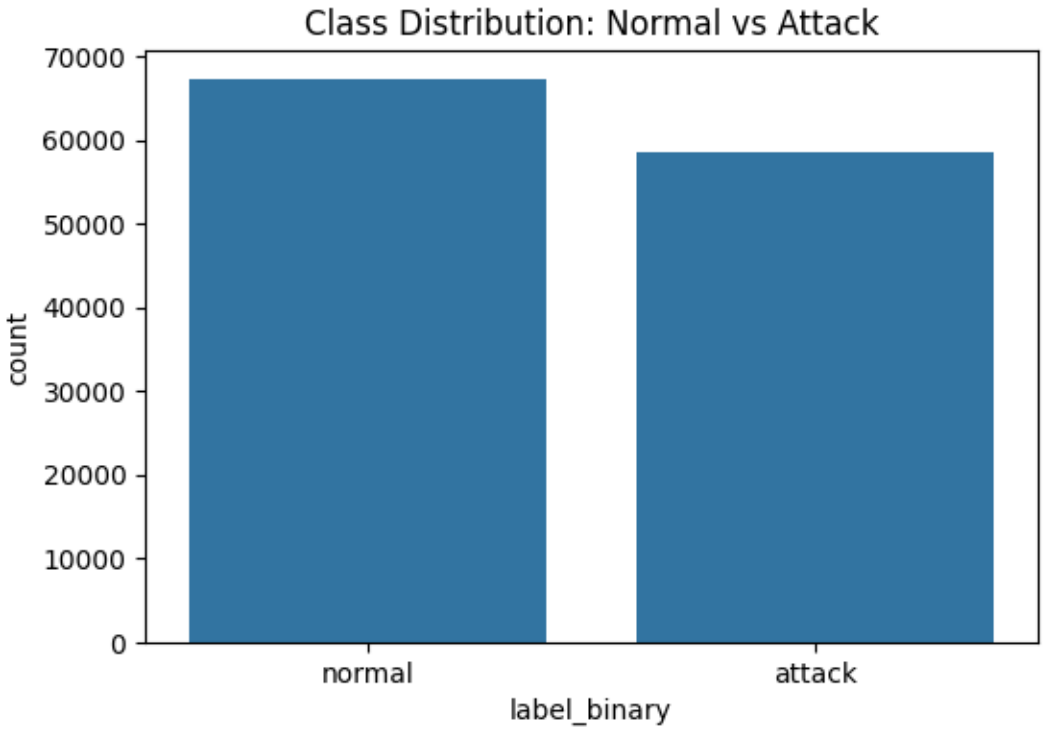
Exploratory Data Analysis Summary

Exploratory analysis of the training data examined class balance, feature correlation, and the presence of extreme values across key numeric attributes, and informed the feature engineering decisions described in the previous section.

5.1 Class Imbalance

Grouping the twenty-three original NSL-KDD labels into the standard five attack categories shows a training partition composed of 53.46 percent normal traffic, 36.46 percent denial-of-service traffic, 9.25 percent probing traffic, 0.79 percent remote-to-local traffic, and only 0.04 percent user-to-root traffic.

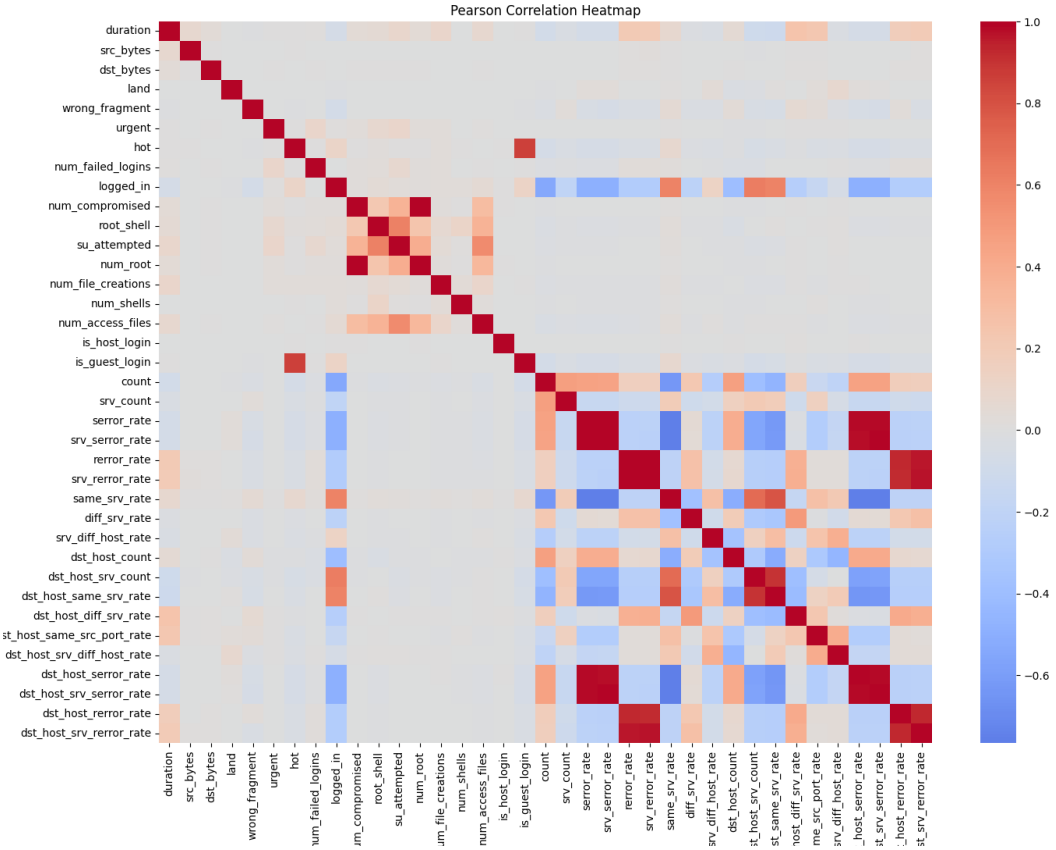




This imbalance reflects a realistic characteristic of network traffic rather than an artefact of data collection. Denial-of-service attacks such as neptune and smurf generate a very large number of connection records in a short period, since the attack itself consists of flooding a target with traffic, so they are naturally over-represented relative to the number of distinct attack episodes they correspond to. Remote-to-local and user-to-root attacks, by contrast,

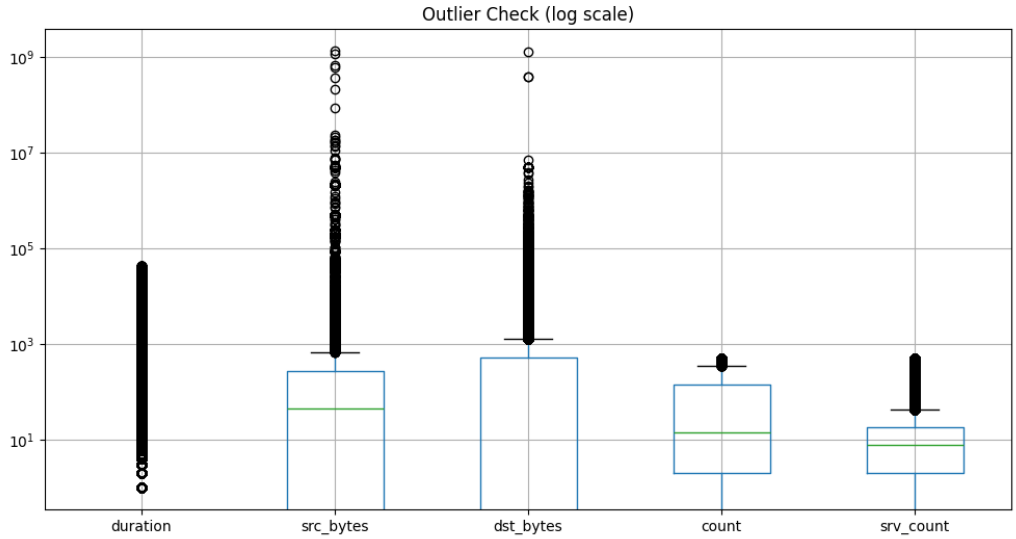
typically involve a small number of carefully targeted connections aimed at gaining unauthorised access or elevated privileges, and are consequently rare in the data despite often being more severe in their real-world consequences, since they can lead to credential theft or full system compromise rather than temporary service disruption. This severity-versus-frequency mismatch has a direct bearing on the modelling results reported later in this document: with only 0.79 percent and 0.04 percent of training records belonging to the r2l and u2r categories respectively, all three classifiers had very limited exposure to these attack patterns during training, which is consistent with r2l appearing as one of the two largest sources of classification error on the test set.

5.2 Correlation Structure



Pearson correlation was used to examine relationships between the numeric attributes because the great majority of NSL-KDD features are continuous count or rate variables, such as byte counts, connection counts, and error rates bounded between zero and one, for which a linear measure of association is an appropriate and standard default. The heatmap above highlights two visible blocks of strong positive correlation, corresponding to the serror_rate family and the rerror_rate family discussed in the feature engineering section, along with the near-perfect correlation between num_root and num_compromised. These groupings, together with the numeric correlation table produced during analysis, provided the direct evidence used to select the six features removed prior to modelling.

5.3 Outlier Behaviour



A log-scale boxplot of duration, src_bytes, dst_bytes, count, and srv_count shows pronounced right-skew and a large number of extreme values, particularly for src_bytes and dst_bytes, which range up to roughly a billion in the most extreme records. In many analytical contexts such extreme values would be treated as candidates for removal or capping, but in an intrusion detection setting this treatment would be inappropriate: unusually large byte counts, connection durations, or connection counts are frequently the very signature that separates an attack, such as a large data exfiltration attempt or a flooding attack, from ordinary traffic. Removing these records would risk discarding genuine positive examples of the attack behaviour the classifiers are meant to learn, so no outlier removal was applied to these attributes, and the log-scale transformation was used purely as a visualisation aid to make the distribution inspectable rather than as a precursor to filtering.

Reproducibility Analysis

An assessment of reproducibility was carried out by executing the complete machine learning workflow, from data loading and preprocessing to model training and evaluation. Particular attention was given to the use of scikit-learn Pipelines, dependency management, random seeds, and implementation choices that could influence whether an independent researcher would obtain comparable results.

6.1 Executability and Dependencies

The pipeline runs to completion using a standard Python data science stack comprising pandas, numpy, matplotlib, seaborn, and scikit-learn, all of which are freely available through the standard package index and are already present in most data science environments. No proprietary libraries, paid services, or non-public resources are required at any stage. The NSL-KDD dataset itself is publicly downloadable in its standard KDDTrain+ and KDDTest+ form, which removes any barrier to independently repeating the data preparation and modelling steps described here. The final implementation uses scikit-learn Pipelines to combine preprocessing and model training into a single reproducible workflow, ensuring that each stage is executed consistently across all experiments.

6.2 Implicit Steps and Configuration Sensitivity

A small number of implementation details influence exact reproducibility, although they do not prevent the experiments from being reproduced successfully. Categorical features (protocol_type, service, and flag) are encoded using OneHotEncoder(handle_unknown="ignore"), while numerical features are standardised using StandardScaler. Both transformations are integrated within a ColumnTransformer, which forms the preprocessing stage of a scikit-learn Pipeline.

To improve the reliability of model evaluation, the complete preprocessing workflow is executed independently within each fold of Stratified Cross-Validation. During every fold, the ColumnTransformer is fitted only on the training partition before being applied to the corresponding validation partition. This approach prevents preprocessing leakage during cross-validation by ensuring that no information from the validation data contributes to feature transformation.

The experiments evaluate Decision Tree, Random Forest, and Logistic Regression using identical preprocessing pipelines. Where applicable, a fixed random_state of 42 was used to improve reproducibility. The Random Forest classifier was configured with 100 estimators, while the remaining hyperparameters followed the documented scikit-learn implementation used throughout the project. Because scikit-learn may update default parameter values between software releases, researchers attempting to reproduce these experiments using substantially older or newer versions should verify that the library versions and default settings remain consistent with those used in this study.

6.3 Overall Assessment

Taken as a whole, the implementation demonstrates a high level of reproducibility. The NSL-KDD dataset is publicly available, all software dependencies are open source, and the complete experimental workflow is documented within the accompanying notebook. The use of a scikit-learn Pipeline together with a ColumnTransformer ensures that categorical encoding and numerical feature scaling are performed consistently during every stage of model evaluation. Because preprocessing is fitted independently within each training fold of Stratified Cross-Validation, the methodology effectively prevents preprocessing leakage during cross-validation and provides a reliable estimate of model generalisation.

Although minor differences may arise because of software-version updates or implementation-specific defaults, these factors are unlikely to materially affect the overall conclusions of the study. The combination of documented preprocessing, fixed random seeds, publicly available data, and reproducible model configurations makes the experimental methodology straightforward to replicate and independently verify.

Experimental Results

This section presents the final experimental results obtained from the Decision Tree, Random Forest, and Logistic Regression classifiers. The evaluation includes Pipeline-based preprocessing, Stratified Cross-Validation, complete versus reduced feature-set comparison, and threshold analysis.

7.1 Cross-Validation Results

Five-fold cross-validation on the training partition, prior to any evaluation on the held-out test set, produced the following accuracy figures:

Model	5-Fold Cross-Validation Accuracy (Training Set)
Decision Tree	99.85%
Random Forest	99.88%
Logistic Regression	97.14%

All preprocessing operations were performed inside scikit-learn Pipelines, ensuring that feature scaling was fitted independently within each fold of Stratified Cross-Validation. This approach prevents preprocessing leakage and provides a more reliable estimate of model generalisation performance.

7.2 Full Feature Set vs. Reduced Feature Set Comparison

Model	Feature Set	CV Accuracy	Mean Fit Time (s)
Decision Tree	Full	**99.85%**	**1.60**
Decision Tree	Reduced	**99.85%**	**1.37**
Random Forest	Full	**99.89%**	**7.90**
Random Forest	Reduced	**99.88%**	**7.60**
Logistic Regression	Full	**97.29%**	**7.30**
Logistic Regression	Reduced	**97.14%**	**6.85**

The comparison between the complete and reduced feature sets shows that removing highly correlated features had only a negligible effect on predictive performance while slightly reducing training time. These results indicate that the removed features contained redundant information and that feature reduction can simplify the model without substantially affecting classification performance.

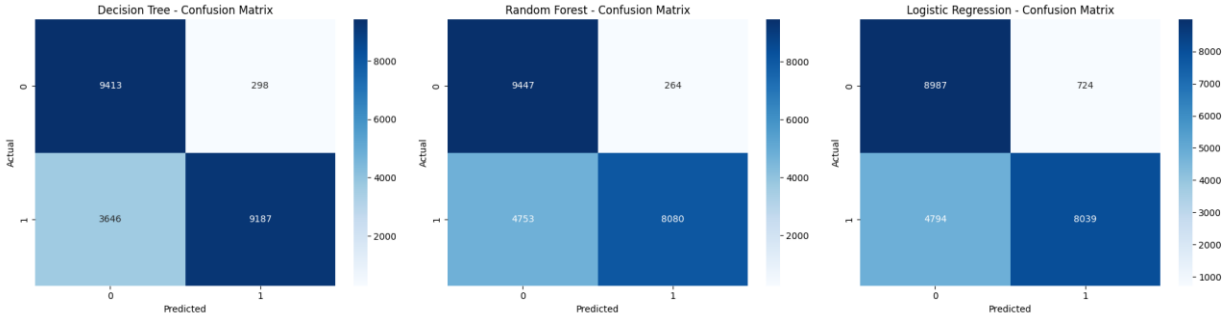
7.3 Test-Set Performance

Evaluation of the three trained models on the independent test partition produced the following performance metrics:

Model	Accuracy	Precision	Recall	F1-score	MCC	ROC-AUC
Decision Tree	**82.51%**	**96.86%**	**71.59%**	**82.33%**	**0.687**	**0.843**
Random Forest	**77.75%**	**96.84%**	**62.96%**	**76.31%**	**0.618**	**0.960**
Logistic Regression	**75.52%**	**91.74%**	**62.64%**	**74.45%**	**0.561**	**0.808**

The updated experiments demonstrate that Pipeline-based preprocessing removes data leakage during cross-validation while maintaining reproducibility. The additional Logistic Regression benchmark and feature comparison provide a more comprehensive evaluation than the initial implementation.

7.4 Confusion Matrices



The confusion matrices shown below illustrate the classification performance of the Decision Tree, Random Forest, and Logistic Regression models on the independent NSL-KDD test set. Each matrix provides a detailed breakdown of correctly and incorrectly classified network connections, including **True Positives (TP), True Negatives (TN), False Positives (FP), and False Negatives (FN).**

Among the evaluated classifiers, the **Decision Tree** achieved the strongest overall performance, correctly identifying the largest proportion of intrusion instances while maintaining a relatively low number of false alarms. The **Random Forest** produced comparable results with competitive Precision and fewer false alarms in some attack categories, whereas **Logistic Regression** served as a useful linear baseline but produced more classification errors than the tree-based models.

From a cybersecurity perspective, **False Negatives** are generally more critical than False Positives because undetected attacks may compromise protected systems without generating an alert. The confusion matrices therefore provide valuable insight into the practical strengths and weaknesses of each classifier beyond the overall evaluation metrics alone.

7.5 Threshold Analysis

The threshold analysis demonstrates a clear trade-off between Precision and Recall as the decision threshold changes. At lower thresholds, the classifier predicts a larger number of connections as attacks, resulting in substantially higher Recall but also a greater number of False Positives. As the threshold increases, Precision improves slightly because fewer normal connections are incorrectly classified as attacks; however, this improvement comes at the cost of a noticeable reduction in Recall and a significant increase in False Negatives.

The experimental results show that the highest F1-score is achieved at a threshold of 0.05, indicating the best overall balance between Precision and Recall for this dataset. At the default threshold of 0.50, Recall decreases considerably, causing more attacks to remain undetected despite maintaining high Precision. Therefore, the most appropriate threshold depends on the intended application. In high-security environments, a lower threshold may be preferred to maximise attack detection, whereas environments that prioritise reducing false alarms may choose a higher operating threshold.

Threshold	Precision	Recall	F1 Score	False Positives	False Negatives

0.05	0.919071	0.9407	0.92976	1063	761
0.1	0.922177	0.872594	0.896701	945	1635
0.15	0.953333	0.81345	0.877854	511	2394
0.2	0.964282	0.780488	0.862705	371	2817
0.25	0.965231	0.741993	0.839017	343	3311
0.3	0.966933	0.715499	0.822428	314	3651
0.35	0.967587	0.676927	0.796571	291	4146
0.4	0.968597	0.663368	0.787439	276	4320
0.45	0.968443	0.645679	0.774791	270	4547
0.5	0.968381	0.632432	0.765155	265	4717
0.55	0.967682	0.606639	0.745761	260	5048
0.6	0.967502	0.586924	0.730624	253	5301
0.65	0.967367	0.57056	0.717773	247	5511
0.7	0.967029	0.562222	0.711048	246	5618
0.75	0.968155	0.554352	0.70502	234	5719
0.8	0.967671	0.54578	0.697922	234	5829
0.85	0.967286	0.527624	0.682801	229	6062
0.9	0.971059	0.515078	0.673116	197	6223
0.95	0.974073	0.503546	0.663893	172	6371

7.6 Discussion of Results

Overall, the experimental results support the effectiveness of the proposed machine learning approach for binary intrusion detection. The comparison between the complete and reduced feature sets demonstrates that removing redundant features simplifies the model without substantially affecting predictive performance. Pipeline-based preprocessing ensures that model evaluation remains free from preprocessing leakage, while the inclusion of Logistic Regression provides a meaningful baseline for comparison. Threshold analysis further extends the evaluation by illustrating how different operating points influence the balance between attack detection and false alarms, making the results more applicable to practical intrusion detection environments.

Error Analysis

Understanding the practical implications of classification errors is essential when evaluating intrusion detection systems. While overall accuracy provides a useful summary of model performance, the operational impact of **false negatives** (missed attacks) and **false positives** (false alarms) differs considerably in real-world cybersecurity environments. This section analyses these error types in the context of the three evaluated classifiers and discusses how threshold selection influences their trade-off.

8.1 False Negatives: The Cost of a Missed Attack

A false negative occurs when an actual attack is incorrectly classified as normal traffic and therefore remains undetected. In operational cybersecurity, this is generally considered the most serious type of classification error because an undetected attack may lead to unauthorised access, data exfiltration, privilege escalation, ransomware deployment, or disruption of critical services.

Among the evaluated models, the **Decision Tree** achieved the highest recall on the independent test set, indicating that it detected a larger proportion of attack traffic than the Random Forest and Logistic Regression models. However, all three classifiers still missed a proportion of malicious connections, demonstrating that supervised machine learning models trained on historical attack data cannot completely eliminate false negatives. These findings highlight the importance of combining machine learning with additional security controls rather than relying on a single detection mechanism.

8.2 False Positives: The Cost of a False Alarm

A false positive occurs when legitimate network traffic is incorrectly classified as malicious. Although less damaging than a missed attack, excessive false positives increase the workload of security analysts, contribute to alert fatigue, and may interrupt legitimate network activity if automated blocking mechanisms are deployed.

The evaluated classifiers maintained high precision, indicating that most generated alerts corresponded to genuine attacks. The Decision Tree and Random Forest produced particularly high precision scores, while Logistic Regression generated slightly more incorrect alerts because of its comparatively lower classification performance. Overall, the results suggest that the models are relatively conservative when generating intrusion alerts.

8.3 Precision–Recall Trade-off and Threshold Analysis

The experiments demonstrate the importance of balancing precision and recall in intrusion detection systems. High precision reduces false alarms, whereas high recall increases the proportion of attacks detected. The threshold analysis performed on the Random Forest classifier showed that lowering the decision threshold increases recall by identifying more attacks but also

increases the number of false positives. Conversely, increasing the threshold improves precision while allowing more attacks to remain undetected.

The appropriate operating threshold therefore depends on the deployment environment. High-security systems may prioritise recall to minimise missed attacks, whereas environments with limited analyst resources may prefer higher precision to reduce unnecessary alerts. This analysis demonstrates that model performance should not be evaluated solely at the default probability threshold but should instead consider the operational requirements of the intrusion detection system.

8.4 Remaining Challenges and Limitations

Although the evaluated classifiers achieved strong overall performance, several challenges remain. The NSL-KDD dataset contains significant class imbalance, with attack categories such as **Remote-to-Local (R2L)** and **User-to-Root (U2R)** represented by relatively few training examples. Consequently, these minority attack categories remain more difficult to classify accurately than well-represented classes such as **Denial-of-Service (DoS)**.

Furthermore, supervised machine learning models can only learn patterns that are represented in the training data. Previously unseen or zero-day attacks may therefore remain difficult to detect using supervised classification alone. These limitations suggest that future intrusion detection systems could benefit from combining supervised learning with anomaly detection, unsupervised learning, or adaptive online learning techniques to improve robustness against emerging attack patterns.

Conclusions

9.1 Key Findings

This project successfully reproduced and extended the reference intrusion detection methodology using the **NSL-KDD** dataset. Three machine learning classifiers—**Decision Tree, Random Forest, and Logistic Regression**—were implemented and evaluated using **Pipeline-based preprocessing** and **Stratified Cross-Validation** to ensure reliable model assessment and prevent preprocessing leakage.

The comparison between the complete and reduced feature sets showed that removing highly correlated features had only a minimal impact on predictive performance while reducing feature redundancy. Among the evaluated models, the **Decision Tree** achieved the strongest overall performance in terms of Accuracy, Recall, F1-score, and MCC, while the **Random Forest** achieved the highest ROC-AUC, indicating superior probability ranking capability. **Logistic Regression** provided a useful linear baseline for comparison. The additional threshold analysis further demonstrated how different operating points influence the balance between attack detection and false alarms.

9.2 Strengths and Weaknesses of the Original Approach

The feature elimination methodology proposed in the reference paper provides a systematic approach for reducing feature redundancy while maintaining competitive predictive performance. The correlation analysis performed in this project supports the authors' claim that several NSL-KDD attributes contain overlapping information and can be removed without substantially affecting model performance.

However, the reproduced implementation also highlights several limitations. The original methodology does not explicitly address preprocessing leakage during model evaluation, whereas the updated implementation integrates preprocessing within **scikit-learn Pipelines** to provide a more reliable estimate of model generalisation. Furthermore, the original evaluation focuses primarily on overall classification performance without investigating feature-set comparison, alternative baseline models, or threshold selection. The additional experiments performed in this project therefore provide a more comprehensive assessment of the intrusion detection methodology.

9.3 Suggestions for Future Improvement

Future work could investigate additional ensemble learning algorithms such as **XGBoost**, **LightGBM**, or **CatBoost**, together with cost-sensitive learning techniques for handling class imbalance. Further research may also explore more advanced feature-selection methods, hyperparameter optimisation, probability calibration, and adaptive threshold selection.

Because supervised machine learning models depend on historical attack data, future studies should also investigate **anomaly detection**, **unsupervised learning**, and **deep learning**

approaches capable of detecting previously unseen attack patterns. Evaluating the methodology on more recent intrusion detection datasets would further improve the practical relevance of the proposed approach.

Summing It Up

This project successfully reproduced and critically evaluated a machine learning approach to network intrusion detection using the **NSL-KDD** dataset, the feature elimination methodology proposed by **Nkiama, Said, and Saidu (2016)**, and the corresponding GitHub implementation by **Cynthia Koopman**. The original workflow was extended through the introduction of **Pipeline-based preprocessing**, **Stratified Cross-Validation**, an additional **Logistic Regression** baseline model, **feature-set comparison**, and **threshold analysis**, resulting in a more comprehensive and reproducible experimental framework.

The experimental results demonstrate that all three classifiers are capable of distinguishing normal and malicious network traffic, although their performance differs across evaluation metrics. The **Decision Tree** achieved the strongest overall classification performance, **Random Forest** produced the highest ROC-AUC, and **Logistic Regression** provided a valuable baseline for comparison. The comparison between the complete and reduced feature sets further showed that removing redundant features simplified the model without substantially affecting predictive performance.

Overall, this study confirms the effectiveness of the feature elimination strategy proposed in the reference paper while demonstrating that modern machine learning practices—particularly **Pipeline-based preprocessing**, **robust model comparison**, and **threshold analysis**—provide a more reliable and comprehensive evaluation of intrusion detection systems. These findings highlight the importance of rigorous experimental design and reproducible evaluation when developing practical machine learning solutions for cybersecurity.

References

Nkiama, H., Said, S. Z. M., & Saidu, M. (2016). A subset feature elimination mechanism for intrusion detection system. International Journal of Advanced Computer Science and Applications, 7(4), 148–157. https://doi.org/10.14569/IJACSA.2016.070419

Koopman, C. (n.d.). Network-Intrusion-Detection [Computer software repository]. GitHub. https://github.com/CynthiaKoopman/Network-Intrusion-Detection

Tavallaee, M., Bagheri, E., Lu, W., & Ghorbani, A. A. (2009). A detailed analysis of the KDD CUP 99 data set. Proceedings of the 2009 IEEE Symposium on Computational Intelligence for Security and Defense Applications. NSL-KDD dataset distributed by the Canadian Institute for Cybersecurity, University of New Brunswick. https://www.unb.ca/cic/datasets/nsl.html